Nightreign Recognizer - Quick Brief for a New Chat

Short companion handoff document. Use together with the longer extensive context PDF when needed.

Status: The project evolved from simple image-similarity recognition into a slot-based, Shifting-Earth-first recognizer with crop refinement, panel normalization, context refinement, debug overlays, monitor-specific investigation, and a GUI that is intended to follow the prototype backend.

1. Original sources and how the project expanded

The starting point was the Nightreign Map Router repository, which provided the map assets, point-of-interest placement logic, map canvas logic, icon mappings, and layout data used to render full reference maps. The recognizer project then grew beyond that source in several directions:

- A custom renderer was created to generate numbered PNG/JPG/WEBP map images, including with-text and no-text outputs.
- A GUI recognizer was created to capture a monitor, crop the in-game map, and display top candidate maps.
- A prototype recognizer was created as a CLI-first debugging environment for faster iteration.
- A slot-based recognition package (.npz) was added, storing Shifting Earth terrain features, per-layout slot labels, and icon templates.
- The project later added zoom/pan, hotkeys, display-folder fallbacks, Windows icon work, debug overlays, crop refinement, and context refinement.

2. Current design idea in one sentence

First classify which Shifting Earth the screenshot belongs to from terrain structure, then examine a catalog of landmark slots for that earth, classify each slot locally, compare the resulting slot pattern to all layouts for that earth, and finally rank candidate layouts.

3. Recognition pipeline

1. Capture monitor or load screenshot.
2. Detect rough map circle. Prefer blue outer ring; otherwise use grayscale Hough fallback.
3. Extract square crop of the map panel.
4. For non-blue-ring crops, run crop refinement by testing nearby shift/scale candidates and keeping the one with the best Shifting Earth objective.
5. Normalize the cropped panel to a canonical square size.
6. Classify Shifting Earth using low-frequency terrain similarity plus gradient similarity.
7. Use the slot catalog for the selected earth and classify each slot locally into Church, Fort, Great Church, Main Encampment, Ruins, Sorcerer's Rise, unknown, or empty.
8. Score every layout in that earth using slot agreement and mismatch penalties.
9. Optionally refine some unknown slots using layout consensus from the top-ranked layouts.
10. Return the ranked layouts and save debug images if requested.

4. Core math

Shifting Earth score:

```
earth_score = 0.65 * low_corr + 0.35 * grad_corr
```

Low-frequency terrain correlation carries more weight because it is more stable across screenshots, while gradient structure still helps separate similar earths.

Crop refinement objective:

```
objective = top_earth_score + 0.35 * margin - 0.12 * black_ratio + 0.03 * edge_mean
```

The refinement pass searches nearby crop candidates and keeps the one with the strongest Shifting Earth confidence, a good gap over the second-best earth, less black-border contamination, and a modest preference for meaningful edge structure.

Runtime search radius for a slot:

```
search_radius = max(slot_radius_norm * side * 1.25, type_based_minimum) * search_radius_scale
```

The slot package stores a nominal radius per slot, but the recognizer enlarges it slightly and enforces a minimum based on the landmark type family. A later debugging version added an optional search-radius scale flag so the runtime radius can be reduced, for example to 70%.

Layout scoring idea:

Occupied expected slots reward the expected label score and penalize competing labels. Empty expected slots penalize strong non-empty evidence. Exact label matches add a bonus; wrong labels and false empties add penalties; unknowns are handled more softly.

Context refinement idea:

Take the top layouts, compute a weighted vote for currently-unknown slots, and only promote an unknown label if both layout consensus and local slot evidence are strong enough.

5. Design decisions that mattered

Decision	Why it was introduced	Result
Canonical panel normalization	The same screenshot gave different results on different monitors due to effective scale differences.	Improved cross-monitor stability.
Crop refinement	Fallback crops on some monitors were slightly off, shifting the fixed slot grid out relative to map patches.	Monitor-2 behavior became much closer to monitor 1 beha
Finite-score sanitization	Earlier runs produced Infinity and NaN values from template matching and later scoring.	Removed numerical instability in ranking.
Context refinement	Too many slots remained unknown after the first pass.	Conservative reduction of unknowns in some cases.
Prototype as authority	The prototype often produced more trustworthy results than the GUI during debugging.	The intended direction became: GUI should follow proto lo

6. Current key files

Authoritative proto	recognize_nightreign_map_proto_v12_authoritative.py
Latest GUI after proto-authoritative fixes	nightreign_map_recognizer_gui_hybrid_v25_proto_authoritative_fix2.py
Debug-plus proto with extra search-radius overlay image	recognize_nightreign_map_proto_v12_debugplus.py
Debug-plus proto with runtime search-radius scaling	recognize_nightreign_map_proto_v12_debugplus_radiusscale_fix.py
Recognition package	nightreign_geom_proto_v4_slots.npz

7. Useful commands

```
Run authoritative proto:
python recognize_nightreign_map_proto_v12_authoritative.py --package
"nightreign_geom_proto_v4_slots.npz" --monitor 2 --debug --json-out "report_monitor2.json"

Run GUI:
python nightreign_map_recognizer_gui_hybrid_v25_proto_authoritative_fix2.py

Run debug-plus proto with orange search-radius circles:
python recognize_nightreign_map_proto_v12_debugplus.py --package
"nightreign_geom_proto_v4_slots.npz" --monitor 2 --debug

Run debug-plus proto at 70% search radius:
python recognize_nightreign_map_proto_v12_debugplus_radiusscale_fix.py --package
"nightreign_geom_proto_v4_slots.npz" --monitor 2 --debug --search-radius-scale 0.7
```

8. Known current issues

- Local slot classification still leaves many slots as unknown or misclassified.
- The GUI and proto have repeatedly drifted apart because logic was copied instead of shared cleanly.
- Changing search-radius scale can change the chosen best earth, so two debug images may not even be drawing the same slot catalog.
- Debug overlays can still be visually confusing unless the chosen earth and slot-count are logged explicitly.

9. Best short explanation to give a new chat

This project started from Nightreign Map Router assets and rendering logic, but it became a separate recognizer project. The recognizer now uses a slot-based approach: classify Shifting Earth from terrain, use an .npz recognition package containing slot metadata and templates, classify local landmark slots, score layouts, and refine some unknowns by consensus. The major recent fixes were canonical panel normalization, crop refinement by earth score, finite-score sanitization, and trying to make the GUI follow the prototype because the prototype was more often correct. The current main questions are about improving local slot classification, reducing unknowns, and keeping GUI and prototype perfectly aligned.